# Institute of Geographical Information Systems

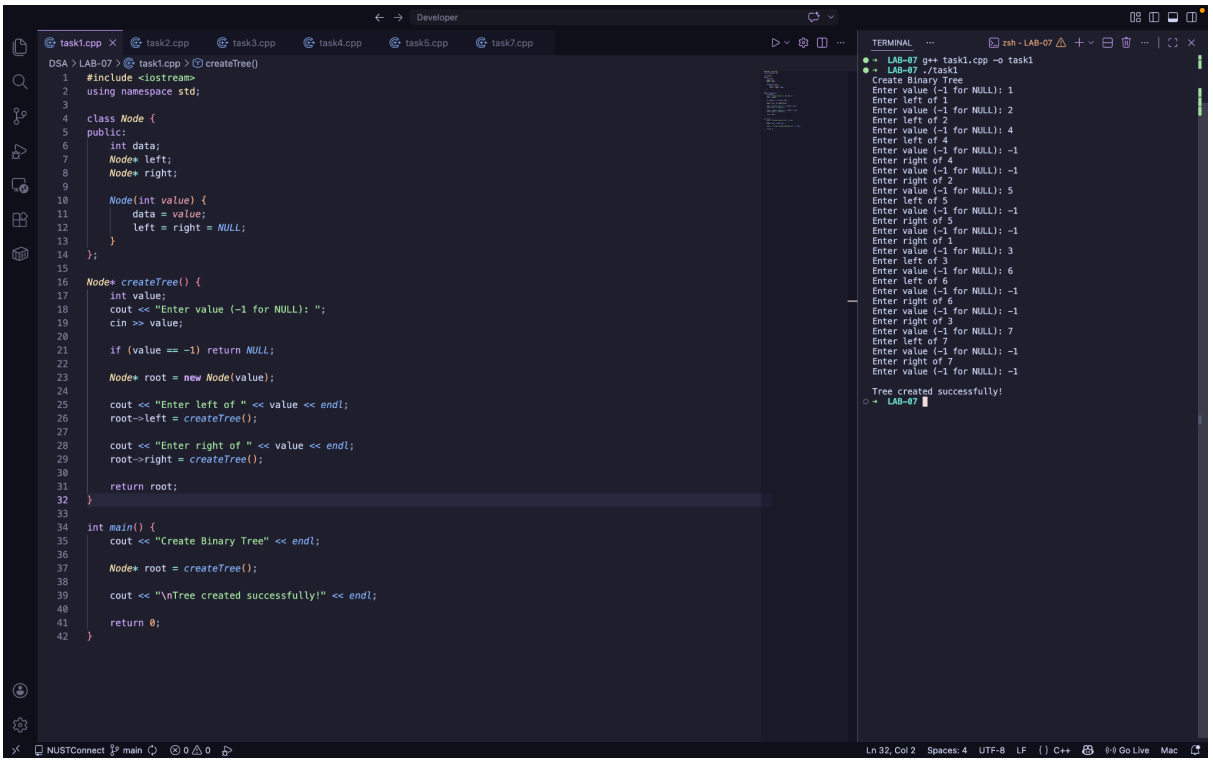
## CS-250 - Data Structures and Algorithms LAB

| | |
|---|---|
| **Semester: Spring 2026** | **Class: SCEE-IGIS - 2024** |
| **Name: Ali Nawaz** | **CMS ID : 00000526123** |
| **Submitted to: Ma'am Alvina Anjum** | **Due Date: April 01, 2026** |

## LAB 7: Exploring Binary Trees in Data Structures & Algorithm

Task 1: Create a Binary Tree Structure

- Define a Node class with data, left, and right child pointers.
- Write a function to create a binary tree from user input (recursive or static).

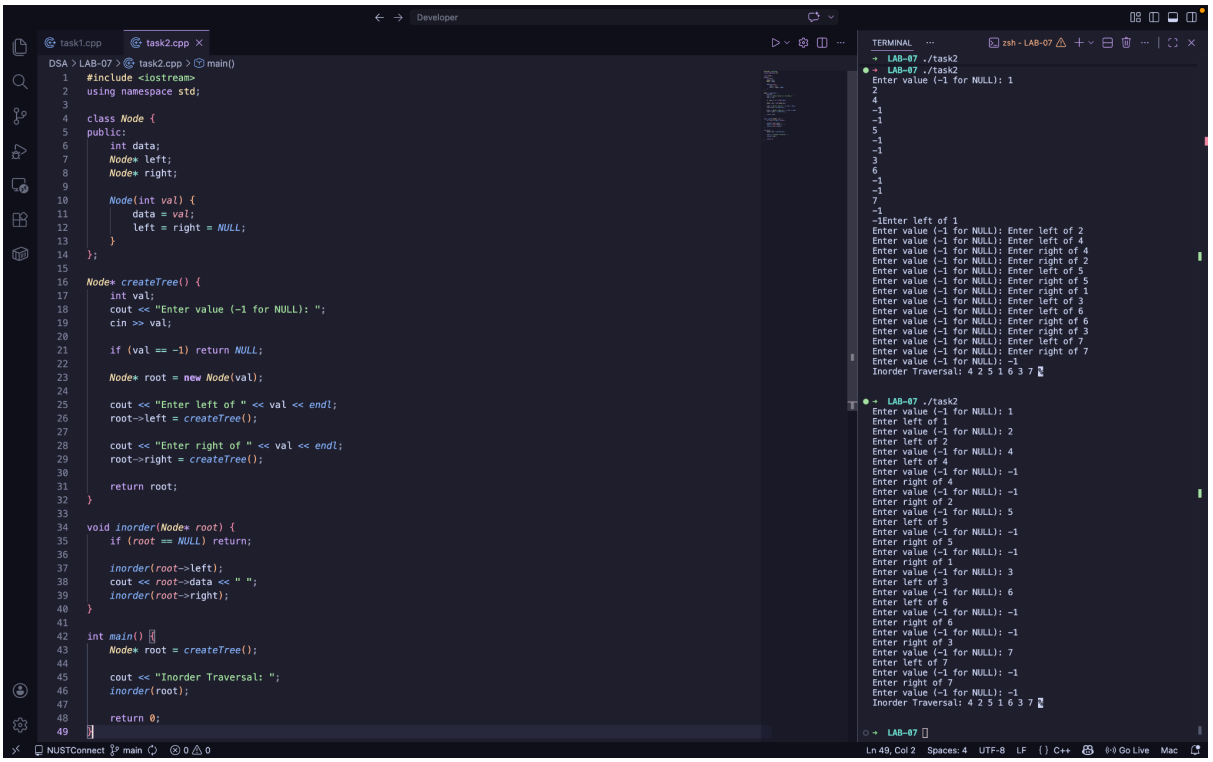


```cpp
#include <iostream>
using namespace std;

class Node {
public:
    int data;
    Node* left;
    Node* right;

    Node(int value) {
        data = value;
        left = right = NULL;
    }
};

Node* createTree() {
    int value;
    cout << "Enter value (-1 for NULL): ";
    cin >> value;

    if (value == -1) return NULL;

    Node* root = new Node(value);

    cout << "Enter left of " << value << endl;
    root->left = createTree();

    cout << "Enter right of " << value << endl;
    root->right = createTree();

    return root;
}

int main() {
    cout << "Create Binary Tree" << endl;

    Node* root = createTree();

    cout << "\nTree created successfully!" << endl;

    return 0;
}
```

```
LAB-07 g++ task1.cpp -o task1
LAB-07 ./task1
Create Binary Tree
Enter value (-1 for NULL): 1
Enter left of 1
Enter value (-1 for NULL): 2
Enter left of 2
Enter value (-1 for NULL): 4
Enter left of 4
Enter value (-1 for NULL): -1
Enter right of 4
Enter value (-1 for NULL): -1
Enter right of 2
Enter value (-1 for NULL): 5
Enter left of 5
Enter value (-1 for NULL): -1
Enter right of 5
Enter value (-1 for NULL): -1
Enter right of 1
Enter value (-1 for NULL): 3
Enter left of 3
Enter value (-1 for NULL): 6
Enter left of 6
Enter value (-1 for NULL): -1
Enter right of 6
Enter value (-1 for NULL): -1
Enter right of 3
Enter value (-1 for NULL): 7
Enter left of 7
Enter value (-1 for NULL): -1
Enter right of 7
Enter value (-1 for NULL): -1

Tree created successfully!
LAB-07
```

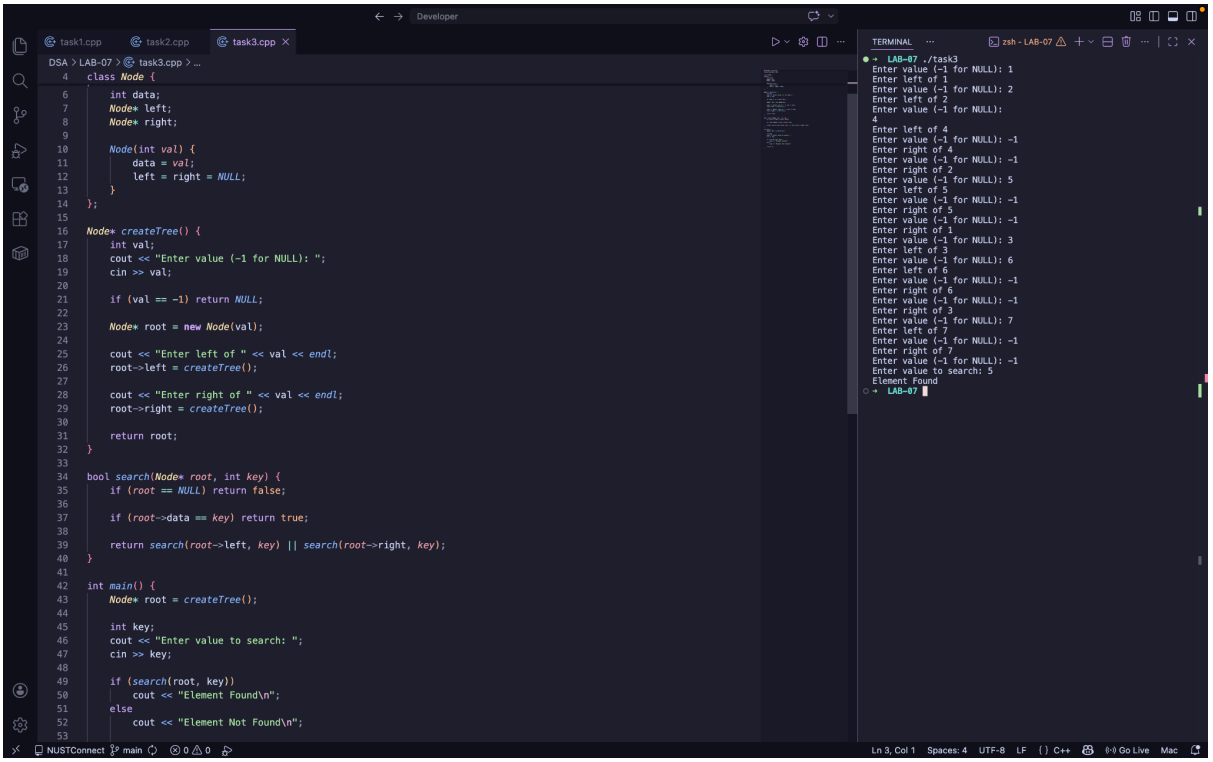
Task 2: Implement Tree Traversals

- Inorder Traversal (Left, Root, Right)



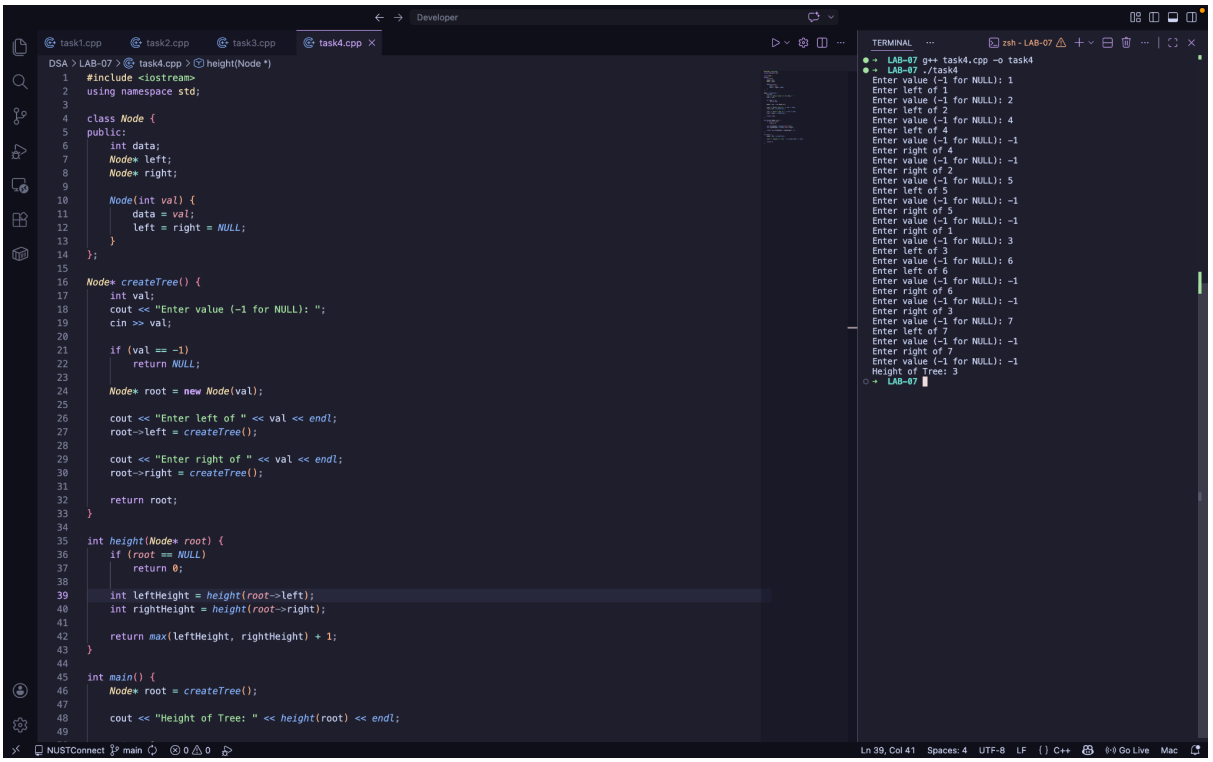

Task 3: Search for an Element

- Write a recursive function to search for an element in the binary tree



Task 4: Calculate Height of Tree

- Implement a function that returns the height of the binary tree.

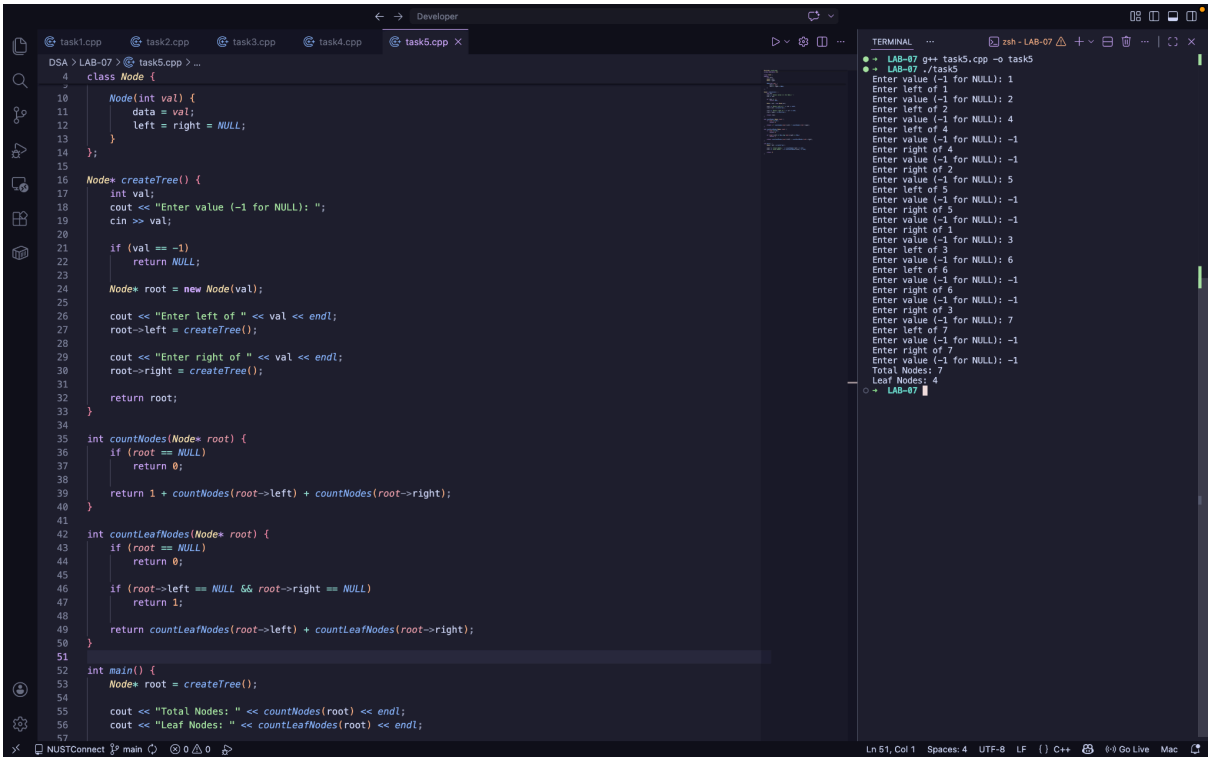


```cpp
#include <iostream>
using namespace std;

class Node {
public:
    int data;
    Node* left;
    Node* right;

    Node(int val) {
        data = val;
        left = right = NULL;
    }
};

Node* createTree() {
    int val;
    cout << "Enter value (-1 for NULL): ";
    cin >> val;

    if (val == -1)
        return NULL;

    Node* root = new Node(val);

    cout << "Enter left of " << val << endl;
    root->left = createTree();

    cout << "Enter right of " << val << endl;
    root->right = createTree();

    return root;
}

int height(Node* root) {
    if (root == NULL)
        return 0;

    int leftHeight = height(root->left);
    int rightHeight = height(root->right);

    return max(leftHeight, rightHeight) + 1;
}

int main() {
    Node* root = createTree();

    cout << "Height of Tree: " << height(root) << endl;
```

```
LAB-07 g++ task4.cpp -o task4
LAB-07 ./task4
Enter value (-1 for NULL): 1
Enter left of 1
Enter value (-1 for NULL): 2
Enter left of 2
Enter value (-1 for NULL): 4
Enter left of 4
Enter value (-1 for NULL): -1
Enter right of 4
Enter value (-1 for NULL): -1
Enter right of 2
Enter value (-1 for NULL): 5
Enter left of 5
Enter value (-1 for NULL): -1
Enter right of 5
Enter value (-1 for NULL): -1
Enter right of 1
Enter value (-1 for NULL): 3
Enter left of 3
Enter value (-1 for NULL): 6
Enter left of 6
Enter value (-1 for NULL): -1
Enter right of 6
Enter value (-1 for NULL): -1
Enter right of 3
Enter value (-1 for NULL): 7
Enter left of 7
Enter value (-1 for NULL): -1
Enter right of 7
Enter value (-1 for NULL): -1
Height of Tree: 3
LAB-07
```

Task 5 (Advanced): Count Leaf Nodes and Total Nodes

- Count the number of leaf nodes and total nodes in the binary tree.



```cpp
class Node {

    Node(int val) {
        data = val;
        left = right = NULL;
    }
};

Node* createTree() {
    int val;
    cout << "Enter value (-1 for NULL): ";
    cin >> val;

    if (val == -1)
        return NULL;

    Node* root = new Node(val);

    cout << "Enter left of " << val << endl;
    root->left = createTree();

    cout << "Enter right of " << val << endl;
    root->right = createTree();

    return root;
}

int countNodes(Node* root) {
    if (root == NULL)
        return 0;

    return 1 + countNodes(root->left) + countNodes(root->right);
}

int countLeafNodes(Node* root) {
    if (root == NULL)
        return 0;

    if (root->left == NULL && root->right == NULL)
        return 1;

    return countLeafNodes(root->left) + countLeafNodes(root->right);
}

int main() {
    Node* root = createTree();

    cout << "Total Nodes: " << countNodes(root) << endl;
    cout << "Leaf Nodes: " << countLeafNodes(root) << endl;
```

```
LAB-07 g++ task5.cpp -o task5
LAB-07 ./task5
Enter value (-1 for NULL): 1
Enter left of 1
Enter value (-1 for NULL): 2
Enter left of 2
Enter value (-1 for NULL): 4
Enter left of 4
Enter value (-1 for NULL): -1
Enter right of 4
Enter value (-1 for NULL): -1
Enter right of 2
Enter value (-1 for NULL): 5
Enter left of 5
Enter value (-1 for NULL): -1
Enter right of 5
Enter value (-1 for NULL): -1
Enter right of 1
Enter value (-1 for NULL): 3
Enter left of 3
Enter value (-1 for NULL): 6
Enter left of 6
Enter value (-1 for NULL): -1
Enter right of 6
Enter value (-1 for NULL): -1
Enter right of 3
Enter value (-1 for NULL): 7
Enter left of 7
Enter value (-1 for NULL): -1
Enter right of 7
Enter value (-1 for NULL): -1
Total Nodes: 7
Leaf Nodes: 4
LAB-07
```

Task 6 (Non-coding): Analyze Tree Traversals

- Given a binary tree diagram or list representation, write down its inorder, preorder, and postorder traversals without writing code. This reinforces understanding of traversal logic.

```
      A
     / \
    B   C
   / \   \
  D   E   F
```

## Traversal Results

- Inorder: Visit left subtree, then root, then right subtree
  **D B E A C F**

- Preorder: Visit root first, then left and right
  **A B D E C F**

- Postorder: Visit left and right first, then root
  **D E B F C A**

Task 7:
In a company's organizational structure represented by a binary tree, each node holds an In a company's organizational structure represented by a binary tree, each node stores an employee's name (string). The CEO is at the root, and each node's left and right child represent that employee’s direct subordinates.
You are required to implement a function to find the Lowest Common Ancestor (LCA) of any two employees in the tree. The LCA of two nodes is the deepest (lowest) node that has both employees as descendants (a node can be a descendant of itself).
Task Requirements:

- Create a Node class/struct that contains:
- string data
- Node* left
- Node* right
- Build the binary tree using static or manual input (hardcoded for simplicity).
- Write a recursive function:

Node* findLCA(Node* root, const string& emp1, const string& emp2);

- Return a pointer to the node that is the lowest common ancestor of emp1 and emp2.

Example Tree:

```
                "Alice"
               /       \
        "Bob"            "Carol"
       /     \           /     \
  "Dave" "Eva"  "Frank" "Grace"
```

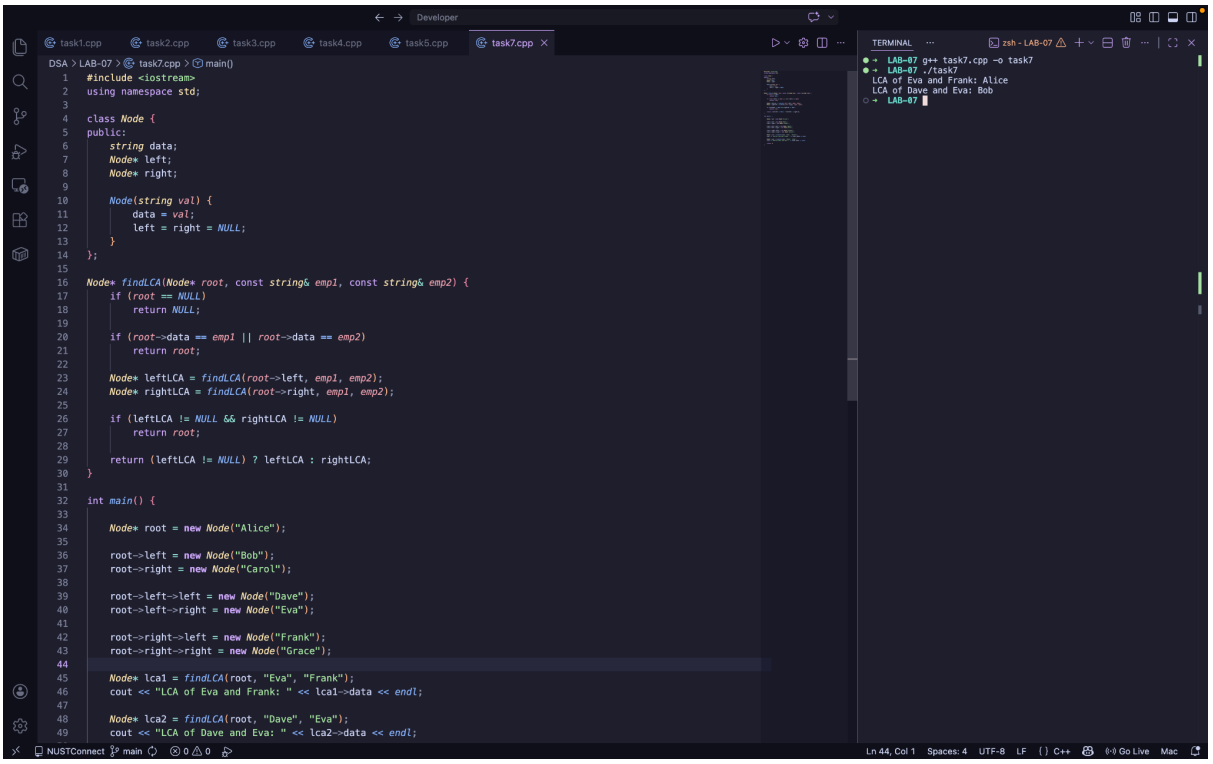
Sample Input/Output:

findLCA(root, "Eva", "Frank");  // Output: Alice

findLCA(root, "Dave", "Eva");  // Output: Bob

Hints:

- Use recursion: if a node matches emp1 or emp2, return it.
- Recurse on left and right subtrees.
- If both left and right return non-null, current node is the LCA.
- If only one side returns a node, propagate it up.



## Difficulties & Challenges Faced

During this lab, I faced several challenges while completing the tasks.

- Since the lecture was conducted **online**, I did not have direct access to the teacher in a physical class. Because of this, it was difficult to ask questions instantly or clear my confusion on the spot.
- The **home environment was not fully suitable for focused study**. There were distractions and it was harder to stay fully concentrated compared to a classroom environment.
- Understanding **recursive concepts** like tree creation, traversal, and LCA was challenging at first because recursion requires step-by-step thinking.
- While implementing the code, I sometimes got confused about **how nodes are connected** (left and right pointers).
- Debugging errors was also difficult because I had to figure out problems on my own without immediate guidance.
- Managing **user input for tree creation** was a bit tricky, especially understanding how recursive input works.

Despite these challenges, I was able to complete the tasks by practicing, reviewing concepts, and trying multiple times.